SKILL BASED
IT TRAINING

AW421 ADVANCED WEB APPLICATION DESIGN II - REACT JS AND REACT NATIVE
PROFESSIONAL DEGREE IN SOFTWARE ENGINEERING
IN-CLASS EXERCISE 4.2

QUESTION 4.2: Validate Score

Create a ReactJS form using **Formik** and **Yup** that asks the user to enter scores for the following assessment components:

- **Project Score**
- **Exercise Score**
- **Homework Score**
- **Presentation Score**

The form must include a hint below the input fields that says:
"**Hint:** Each score must be a number within its allowed range."

Validation Requirements:

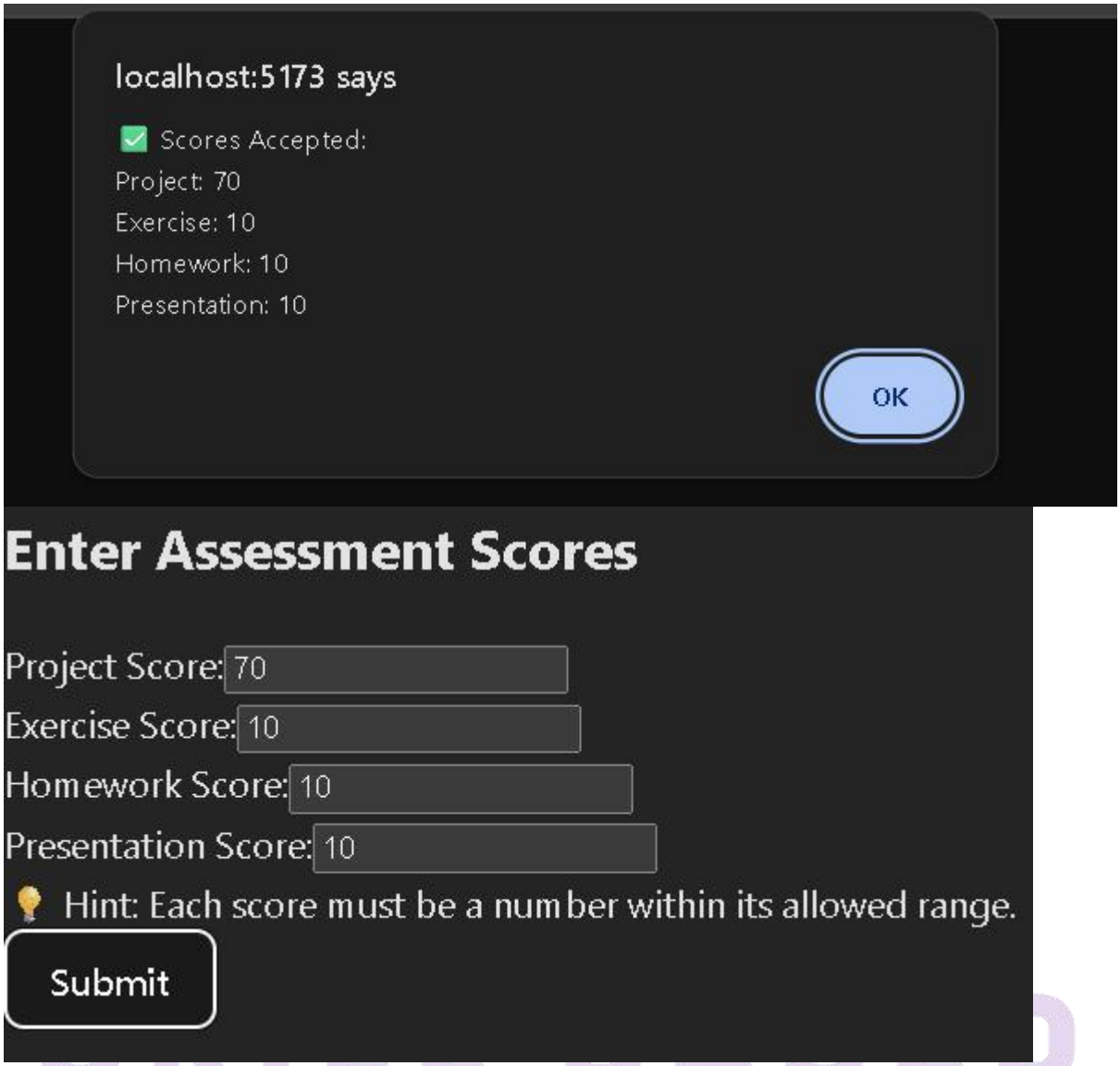
- All fields are **required**.
- All scores must be **numbers**.
- **Project Score:** must be between **0 and 70**.
- **Exercise Score:** must be between **0 and 10**.
- **Homework Score:** must be between **0 and 10**.
- **Presentation Score:** must be between **0 and 10**.
- Display **appropriate error messages** for each invalid case.

Expected Output:

- Shows an error message if any field is empty, non-numeric, or outside its range.
- Accepts valid numeric values such as:
  - Project = 45
  - Exercise = 8
  - Homework = 9.5
  - Presentation = 10

SKILL BASED
IT TRAINING

ANSWER:



ValidateScore.jsx

```
import React from "react";
import { useFormik } from "formik";
import * as Yup from "yup";

function ValidateScore() {
  const formik = useFormik({
    initialValues: {
      project: "",
      exercise: "",
      homework: "",
      presentation: "",
    },
```

SKILL BASED
IT TRAINING

```
    validationSchema: Yup.object({
      project: Yup.number()
        .typeError("❌ Project Score must be a number.")
        .required("❌ Project Score is required.")
        .min(0, "❌ Minimum is 0.")
        .max(70, "❌ Maximum is 70."),
      exercise: Yup.number()
        .typeError("❌ Exercise Score must be a number.")
        .required("❌ Exercise Score is required.")
        .min(0, "❌ Minimum is 0.")
        .max(10, "❌ Maximum is 10."),
      homework: Yup.number()
        .typeError("❌ Homework Score must be a number.")
        .required("❌ Homework Score is required.")
        .min(0, "❌ Minimum is 0.")
        .max(10, "❌ Maximum is 10."),
      presentation: Yup.number()
        .typeError("❌ Presentation Score must be a number.")
        .required("❌ Presentation Score is required.")
        .min(0, "❌ Minimum is 0.")
        .max(10, "❌ Maximum is 10."),
    }),
    onSubmit: (values) => {
      alert(
        `✅ Scores Accepted:\nProject: ${values.project}\nExercise: ${values.exercise}\nHomework:
${values.homework}\nPresentation: ${values.presentation}`
      );
    },
  });
```

```
  return (
    <div className="space">
      <div className="form-container">
        <form onSubmit={formik.handleSubmit} className="score-form">
          <h2>Enter Assessment Scores</h2>
```

```
          {["project", "exercise", "homework", "presentation"].map((field) => (
            <div key={field}>
              <label>
                {field.charAt(0).toUpperCase() + field.slice(1)} Score:
              </label>
              <input
                type="text"
                name={field}
                onChange={formik.handleChange}
                onBlur={formik.handleBlur}
                value={formik.values[field]}
              />
              {formik.touched[field] && formik.errors[field] && (
                <div className="error-message">{formik.errors[field]}</div>
              )}
            </div>
          ))}
```

```
          <div className="hint">
              Hint: Each score must be a number within its allowed range.
```

```
          </div>
```

```
          <button type="submit">Submit</button>
        </form>
      </div>
    </div>
  );
}
```

```
export default ValidateScore;
```

SKILL BASED
IT TRAINING